

PROJECT REPORT

Grievance

Root-Cause Discovery and Resolution Outcome Prediction for Consumer Financial Complaints

Sprint 1: Data Cleaning, Data Quality Analysis and Exploratory Data Analysis

Submitted by: Sivasuriyan V

Abstract:

Consumer financial complaints are a direct signal of where financial products and services fail the people who use them. The Consumer Financial Protection Bureau (CFPB) collects these complaints at scale, but the raw records are inconsistent, partly incomplete and difficult to interpret without systematic preparation. This project, Grievance, aims to discover the root causes behind consumer complaints and, in later sprints, to predict resolution outcomes.

Sprint 1, reported here, covers data cleaning, data quality analysis and exploratory data analysis (EDA) on the CFPB Consumer Financial Complaints dataset using Python, pandas and matplotlib. The workflow creates raw and clean copies of the data, removes duplicates, standardizes column names, converts data types, cleans text fields, treats missing values, and engineers response-time and year features. The cleaned data is then analyzed by product, issue, sub-issue, company and state to identify where complaint volume concentrates. The output is a validated, analysis-ready dataset and a set of baseline insights that form the foundation for NLP-based root-cause discovery and outcome prediction in subsequent sprints.

Keywords: *consumer complaints, CFPB, exploratory data analysis, data cleaning, data quality, root-cause analysis, Python.*

1. Introduction

Banks, credit card issuers, mortgage servicers and other financial institutions handle millions of customer interactions every year. When those interactions go wrong, consumers can file a complaint with the Consumer Financial Protection Bureau, which routes it to the company and records the company's response. Over time this process builds a large public record of what consumers experience and how companies react.

This record is valuable to regulators, compliance teams, product owners and researchers. It shows which products generate the most dissatisfaction, which issues recur, which companies receive the most complaints, and how outcomes differ across cases. However, extracting that value requires the data to be trustworthy. Dates must be parsed correctly, categories must be consistent, missing fields must be handled deliberately, and derived measures such as response time must be validated.

The Grievance project is structured as a multi-sprint effort. Sprint 1 establishes the data foundation: a cleaned dataset, a documented quality assessment, and an exploratory view of complaint patterns. This report documents that work in the order it was carried out.

2. Existing Problem

The current handling and analysis of consumer financial complaints faces the following practical problems:

- **Scale of the data.** Complaint volumes are large and growing, which makes manual review slow and inconsistent.

- **Incomplete and inconsistent records.** Fields such as sub-issue, company public response and tags are frequently blank, and text fields contain stray whitespace and mixed casing, which distorts counts and groupings.
- **Labels describe symptoms, not causes.** Complaints are categorized by product and issue, but these labels describe what was reported, not why it happened. The underlying root causes are not visible from the category counts alone.
- **No consolidated view.** Without a prepared dataset, there is no single reliable view of complaint volume by product, company and geography, nor of how long it takes for a complaint to reach the company.
- **Data quality risks.** Date columns arrive as text, ZIP codes can be malformed, and derived values such as response time can be negative if dates are entered incorrectly. Analysis built on unchecked data risks misleading conclusions.
- **Limited outcome analysis.** Resolution outcomes (for example, whether a consumer receives relief) are not systematically examined against complaint characteristics, so patterns in how complaints are resolved remain unexplored.

3. Problem Statement

Consumer financial complaint data is large, partially incomplete and inconsistently formatted, which prevents regulators and financial institutions from reliably identifying the root causes of complaints and understanding how they are resolved. There is a need for a systematic, reproducible process that cleans and validates the complaint data, measures its quality, and explores complaint patterns across products, issues, companies and states, so that the dataset can support root-cause discovery and resolution outcome prediction.

4. Objectives

4.1 Overall Objective

To build a clean, validated and well-understood dataset of consumer financial complaints and use exploratory analysis to uncover the patterns that drive complaint volume and resolution outcomes.

4.2 Sprint 1 Objectives

1. Clean the dataset: remove duplicates, standardize column names, convert data types and clean text fields.
2. Assess data quality: quantify missing values, validate complaint IDs and check for invalid response times.
3. Treat missing values with clear, documented rules instead of silent deletion.
4. Engineer features that support analysis, namely response time in days and complaint year.
5. Perform exploratory analysis of complaints by product, issue, sub-issue, company and state.
6. Summarize resolution outcome and response patterns as a baseline for later modelling.

7. Document findings as business insights and hand over a final cleaned dataset.

5. Scope of Sprint 1

In scope: data loading, cleaning, quality analysis, feature engineering, descriptive statistics, univariate frequency analysis with visualization, and export of the cleaned dataset.

Out of scope (planned for later sprints): NLP on complaint narratives, topic modelling, resolution outcome prediction, company-level relief gap scoring, model explainability and dashboard development.

6. Dataset Description

The project uses the Consumer Financial Complaints dataset published by the CFPB. Each row is one consumer complaint. After column-name standardization, the fields used in Sprint 1 are listed below.

Field	Description
complaint_id	Unique identifier of the complaint
date_received	Date the CFPB received the complaint
date_sent_to_company	Date the complaint was forwarded to the company
product / sub_product	Financial product the complaint relates to, and its sub-category
issue / sub_issue	Problem reported by the consumer, and its sub-category
company	Company the complaint was filed against
company_public_response	Optional public statement made by the company
company_response_to_consumer	How the company closed or responded to the complaint
state / zip_code	Consumer location
tags	Special consumer categories attached to the complaint
submitted_via	Channel through which the complaint was submitted
timely_response	Whether the company responded on time

Two engineered fields are added during cleaning: `response_days` (days between receipt and forwarding to the company) and `year` (year of receipt).

7. Tools and Technologies

Tool	Use in this project
Python	Core programming language for the entire workflow
pandas	Data loading, cleaning, transformation and aggregation
NumPy	Numerical support
matplotlib	Visualization of missing values and complaint distributions
Jupyter Notebook	Interactive development and documentation of the analysis

8. Methodology: Step-by-Step EDA Process

The analysis follows a fixed order so that every transformation is traceable: load, back up, inspect, clean, validate, engineer features, analyze, verify and save. Each step below states its purpose, the action taken and the reason for the decision.

Step 1: Import Libraries and Load the Dataset

Purpose: Prepare the environment and bring the complaint data into memory.

Action: Import NumPy, pandas and matplotlib, then read the CFPB CSV file with pandas and print the dataset shape to confirm a successful load.

Rationale: Confirming the shape immediately catches path errors and incomplete loads before any processing begins.

Step 2: Create Raw and Clean Copies

Purpose: Protect the original data.

Action: Create `raw_data` and `clean_data` as separate copies of the loaded dataset. All transformations are applied only to `clean_data`.

Rationale: Keeping an untouched raw copy allows any cleaning decision to be audited or reversed.

Step 3: Initial Inspection

Purpose: Understand the structure of the data before changing it.

Action: Preview the first rows, review column names and data types, count rows and columns, count nulls per column and generate descriptive statistics for all columns.

Rationale: Inspection reveals which columns need type conversion, which contain missing values and what needs cleaning.

Step 4: Duplicate Analysis

Purpose: Ensure each complaint is counted once.

Action: Count fully duplicated rows and drop them only if any exist.

Rationale: Duplicates inflate counts for products, issues and companies and would bias every later analysis.

Step 5: Standardize Column Names

Purpose: Make column names consistent and easy to reference in code.

Action: Strip whitespace, convert to lowercase, replace spaces and hyphens with underscores and remove question marks (for example, "Date received" becomes `date_received`).

Rationale: Uniform snake_case names prevent key errors and make the code readable.

Step 6: Data Type Conversion

Purpose: Store each field in the type that matches its meaning.

Action: Convert `date_received` and `date_sent_to_company` to datetime using `errors="coerce"`, so invalid values become missing instead of raising errors. Convert `zip_code` to numeric, fill unparseable values with 0 and cast to integer.

Rationale: Correct types are required for date arithmetic and for reliable aggregation.

Step 7: Text Cleaning

Purpose: Remove formatting noise from categorical text fields.

Action: Cast `product`, `sub_product`, `issue`, `sub_issue`, `company_public_response`, `company`, `state`, `tags`, `submitted_via`, `company_response_to_consumer` and `timely_response` to the string type and strip leading and trailing whitespace.

Rationale: Hidden whitespace causes identical categories to be counted as different values.

Step 8: State Cleaning

Purpose: Make state codes uniform.

Action: Convert the state column to uppercase and strip whitespace, then inspect the top state counts.

Rationale: Mixed casing would split one state into several categories.

Step 9: Complaint ID Validation

Purpose: Verify that `complaint_id` works as a unique key.

Action: Compare the total record count with the number of unique complaint IDs and count duplicate IDs.

Rationale: A unique identifier is essential for joining data later and confirms that no complaint is repeated.

Step 10: Missing Value Analysis

Purpose: Measure how much data is missing and where.

Action: Compute the missing count and missing percentage for every column and plot the percentage for columns that have missing values as a bar chart.

Rationale: Quantifying missingness guides the treatment strategy for each column instead of applying one rule to all.

Step 11: Missing Value Treatment

Purpose: Handle missing values without discarding records.

Action: Fill `company_public_response` with "Not Provided", `tags` with "No Tag", `state` with "Unknown" and `sub_issue` with "Not Specified". Re-check null counts afterwards.

Rationale: In these fields a blank usually means that no value was given, so a meaningful label preserves the record and keeps the absence visible as its own category.

Step 12: Feature Engineering

Purpose: Create fields that support time-based and response analysis.

Action: Create `response_days` as the difference in days between `date_sent_to_company` and `date_received`, count any negative values, and create `year` from `date_received`.

Rationale: Response time is a key operational measure, and the negative-value check validates date integrity.

Step 13: Final Data Quality Check

Purpose: Confirm the dataset is ready for analysis.

Action: Report the row count, column count, duplicate rows, total missing values and the number of negative response-day records, then review data types and descriptive statistics for numeric and string columns.

Rationale: A final check proves the cleaning steps worked and gives a documented quality baseline.

Step 14: Product Analysis

Purpose: Identify which financial products generate the most complaints.

Action: Count complaints by product and plot the top 10 as a bar chart.

Rationale: Product-level volume shows where consumer problems are concentrated.

Step 15: Issue Analysis

Purpose: Identify the most common problems reported.

Action: Count complaints by issue and plot the top 15 as a horizontal bar chart.

Rationale: Issue counts give the first view of root causes and prioritize what to investigate further.

Step 16: Sub-Issue Analysis

Purpose: Add detail beneath each issue.

Action: Count complaints by sub-issue and plot the top 15 as a horizontal bar chart.

Rationale: Sub-issues refine the issue categories into more specific failure points.

Step 17: Company Analysis

Purpose: Identify which companies receive the most complaints.

Action: Count complaints by company and plot the top 10 as a bar chart.

Rationale: Company volume highlights where complaints cluster. Raw counts should later be normalized by company size or customer base.

Step 18: State Analysis

Purpose: Understand geographic distribution.

Action: Count complaints by state and plot the top 15 as a bar chart.

Rationale: Geographic patterns can reflect population, market presence or regional service problems.

Step 19: Data Quality Summary and Final Verification

Purpose: Document the final state of the dataset in one place.

Action: Build a summary table of total records, total columns, duplicate rows, missing values, unique complaint IDs and negative response days, and print a final verification report.

Rationale: A single summary table makes the quality of the final dataset easy to review and report.

Step 20: Save the Cleaned Dataset

Purpose: Create a reusable, analysis-ready output.

Action: Export `clean_data` to `Consumer_Financial_Complaints_Cleaned.csv` without the index.

Rationale: Later sprints start from this file, so cleaning is performed once and reused.

Summary of Cleaning Decisions

Issue found	Treatment	Effect
Duplicate rows	Dropped if present	Prevents inflated counts
Inconsistent column names	Standardized to snake_case	Consistent, error-free referencing
Dates stored as text	Converted with errors=coerce	Enables date arithmetic

Issue found	Treatment	Effect
Malformed ZIP codes	Coerced to numeric, filled with 0	Uniform integer field
Whitespace and casing in text	Stripped; state uppercased	Merges duplicate categories
Missing categorical values	Filled with explicit labels	Keeps all records
No response-time field	response_days engineered	Enables response analysis

9. Project Code Structure

The notebook logic is also organized as a modular Python package so that each stage can be run independently or as a pipeline.

File	Responsibility
Python/data_loading.py	Loads the dataset, creates raw and clean copies, and inspects structure
Python/data_cleaning.py	Runs the full cleaning pipeline and saves the cleaned dataset
Python/exploratory_analysis.py	Missing-value summary, descriptive statistics, frequency counts and the data quality summary
Python/data_visualization.py	Generates the missing-value chart and the product, issue, sub-issue, company and state charts

10. Results and Business Insights

The cleaned dataset passed the final quality check: no duplicate rows, no missing values after treatment, unique complaint IDs and no negative response-day records. The tables below are to be completed with the exact values produced by the notebook outputs.

11. Conclusion

Sprint 1 converted the raw CFPB complaint data into a clean, validated and documented dataset. Duplicates were handled, names and types were standardized, text fields were cleaned, missing values were treated with explicit rules, and response time and year were engineered.

Exploratory analysis then mapped complaint volume across products, issues, sub-issues, companies and states, giving a clear baseline of where consumer problems concentrate.

This foundation allows the next sprints to focus on what the data cannot yet answer: why complaints occur and how they are resolved. The cleaned dataset, modular code and documented decisions make that work reproducible.